

VR Chemistry Lab – WebSocket API Reference

This document is the contract between the VR client and the simulation server: message formats, methods, error codes, concurrency rules, and session lifecycle. For a description of the current Godot reference client and the Python reference server, see the companion document *VR Chemistry Lab – System Architecture*. For extending the front-end see *VR Chemistry Lab – Handbook*.

Contents

1	Transport & Conventions	2
2	Request (Client → Server)	2
3	Response (Server → Client)	2
4	Concurrency & Request Constraints	3
5	Error Codes	3
6	Method Catalog	4
7	Session Lifecycle	5
8	Example Exchange	6

1 Transport & Conventions

- Messages are exchanged as complete, individual JSON text frames over a single persistent WebSocket connection.
- Every reaction group is one *session*, identified by a client-chosen `reactionInstanceId`. A connection can multiplex several concurrent sessions.
- All JSON keys use `camelCase`.
- Units: temperature in Kelvin, pressure in Pascal, time in seconds (request `targetTime`) except the locally tracked client clock, which uses milliseconds; amount of substance in mol.
- Every request is answered with exactly one response (see Section 3); the protocol does not define server-initiated (unsolicited) messages.

2 Request (Client → Server)

Field	Type	Required	Description
<code>methodName</code>	string	yes	see Method Catalog (Section 6)
<code>reactionInstanceId</code>	string	yes	session key, chosen by the client
<code>reactionIds</code>	dict	yes	maps chemical name → client-side instance id; echoed back unchanged in the response
<code>whatReaction</code>	string	init only	reaction name, e.g. "NaCl2" – must be one of the values listed in Section ??
<code>temperature</code>	float	init only	starting temperature T0
<code>amountInMol</code>	dict null	init, optional	e.g. {"Na": 0.4, "Cl2": 0.04} – overrides the simulation's default starting amounts
<code>targetTime</code>	float	run/heat-ing calls	target time for <code>runUntilTargetTime</code>

3 Response (Server → Client)

Every response — success, ack, or error — has exactly this shape:

```
{
  "success": true,
  "reactionInstanceId": "abc123",
  "reactionIds": { "Na": 3, "Cl2": 1 },
  "data": { "dataBlockList": [ /* ... */ ] },
  "error": null
}
```

Field	Type	Description
success	bool	false on any error
reactionInstanceId	string null	echoed from the request
reactionIds	dict null	echoed from the request
data	dict null	null on error. Otherwise: {"dataBlockList": [...]} (init/run methods) or {"sessionClosed": bool} (close Session)
error	dict null	null on success. Otherwise {"code": "...", "message": "..."}, see Error Codes (Section 5)

Not every entry inside `dataBlockList` represents a playback step: acknowledgements such as `{"initSuccess": true, ...}` carry no `timestamp` field. Consumers should treat only entries that *do* carry a `timestamp` as simulation steps to be played back.

4 Concurrency & Request Constraints

- **One in-flight request per session.** A session must not have more than one outstanding request at a time; a client must wait for the response before sending the next request for the same `reactionInstanceId`. Sessions are otherwise fully independent of one another.
- **Connection must be ready before init.** A client must not send `init` before its WebSocket connection has completed its handshake.
- **init is mandatory before any other method.** Calling `runUntilTargetTime` or either heating method before a successful `init` results in `SIM_NOT_INITIALIZED`.
- **closeSession is always accepted.** Unlike other methods, `closeSession` may be sent even while another request for the same session is outstanding, so that teardown is never blocked. It is a no-op (`sessionClosed: false`) if no session exists for the given `reactionInstanceId`.
- **Switching reactions on an existing reactionInstanceId** requires closing the current session first, in particular when the new `whatReaction` is handled by a different simulation engine than the previous one.

5 Error Codes

Code	Meaning
MISSING_REACTION_INSTANCE_ID	request sent without <code>reactionInstanceId</code>
SIM_NOT_INITIALIZED	a method other than <code>init</code> was called before <code>init</code> succeeded
UNKNOWN_METHOD	<code>methodName</code> doesn't match any known method
UNKNOWN_REACTION	<code>whatReaction</code> has no matching simulation on the server
NULL_OUTPUT	internal server handler returned no result (should not happen in practice)
INTERNAL_ERROR	unexpected server-side exception (e.g. a solver error, malformed input)

6 Method Catalog

methodName	Precondition	Uses from request	data on success	Possible errors
init	–	whatReaction, temperature, amountInMol?	{"dataBlockList": [{"initSuccess": true, "temperature": T0}]}	UNKNOWN_REACTION
runUntilTargetTime	session must be initialized	targetTime	{"dataBlockList": [{...step}]}	SIM_NOT_INITIALIZED
startHeatingAndRunUntilTargetTime	session must be initialized	targetTime	same as above; from now on every following call ramps the temperature up linearly until stopHeating... (STANDART_HEATING_POWER = 7 K/s)	SIM_NOT_INITIALIZED
stopHeatingAndRunUntilTargetTime	session must be initialized	targetTime	same as above; heating is turned off	SIM_NOT_INITIALIZED
closeSession	–	–	{"sessionClosed": bool} (true if a session existed and was removed)	–
(unknown value)	–	–	–	UNKNOWN_METHOD

Step object inside dataBlockList (common fields):

Field	Description
timeStamp	simulation time in s
temperature	reactor temperature in K
pressure	reactor pressure in Pa
amount_of_<reactant/product>	reaction-specific – surface coverage (0–1) or normalized gas concentration; names and count vary per whatReaction (see Section ??)

7 Session Lifecycle

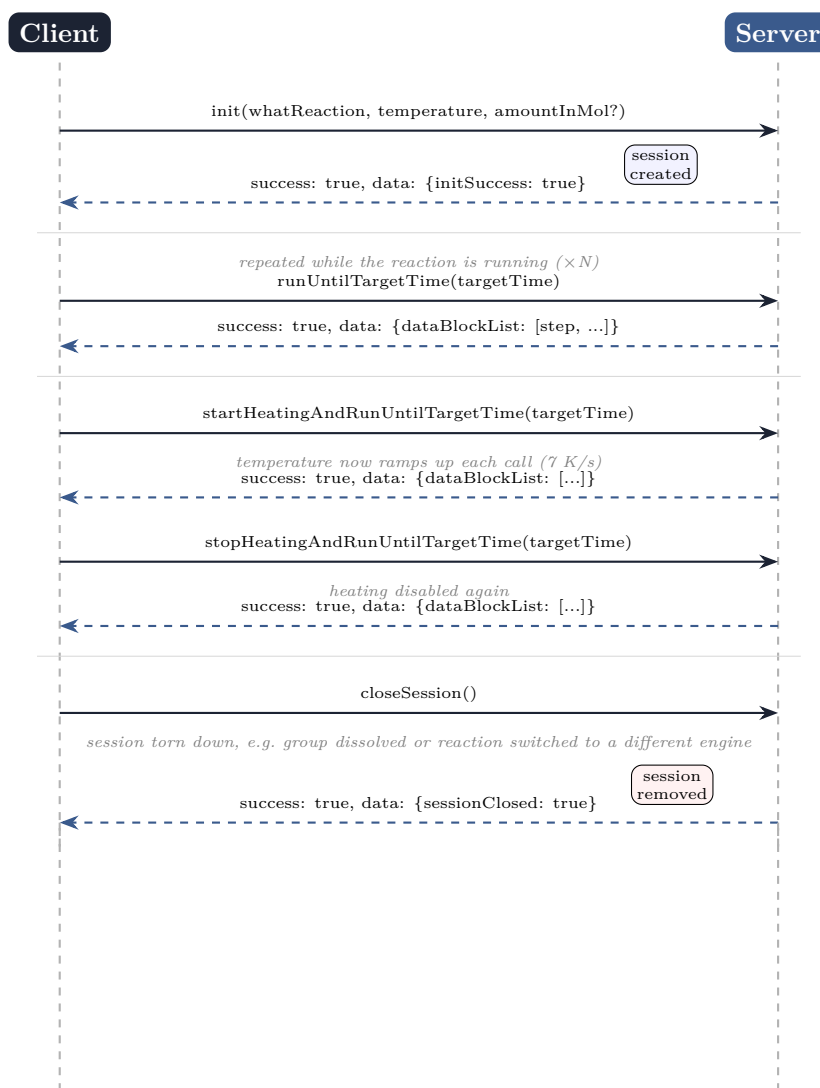


Figure 1: Session lifecycle: request/response sequence between client and server.

8 Example Exchange

Initializing a NaCl2 session with a custom starting amount of sodium, then requesting one simulation step:

```
// Client -> Server
{
  "methodName": "init",
  "reactionInstanceId": "grp_7f3a",
  "reactionIds": { "Na": 3, "Cl2": 1 },
  "whatReaction": "NaCl2",
  "temperature": 298.15,
  "amountInMol": { "Na": 0.4, "Cl2": 0.04 }
}

// Server -> Client
{
  "success": true,
  "reactionInstanceId": "grp_7f3a",
  "reactionIds": { "Na": 3, "Cl2": 1 },
  "data": { "dataBlockList": [ { "initSuccess": true, "temperature": 298.15 } ] },
  "error": null
}

// Client -> Server
{
  "methodName": "runUntilTargetTime",
  "reactionInstanceId": "grp_7f3a",
  "reactionIds": { "Na": 3, "Cl2": 1 },
  "targetTime": 1.0
}

// Server -> Client
{
  "success": true,
  "reactionInstanceId": "grp_7f3a",
  "reactionIds": { "Na": 3, "Cl2": 1 },
  "data": { "dataBlockList": [ {
    "timeStamp": 1.0, "temperature": 298.15, "pressure": 101325,
    "amount_of_Na": 0.91, "amount_of_NaCl": 0.09, "amount_of_Cl2": 0.97
  } ] },
  "error": null
}
```